

Leetcode

刷题

[最长连续子序列](#)

[最短路径](#)

[背包问题](#)

困难

[计算最小路径和并返回路径](#)

[接雨水问题](#)

中等

[最长连续子序列长度](#)

[53. 最大子序和](#)

[最长回文子串](#)

简单

[2Sum、3Sum、4Sum问题](#)

最长连续子序列

```
1 package com.alibaba.alisc.lemon.model.algo.dp;
2
3 import java.util.Arrays;
4
5 /**
6  * 动态规划求解:
7  * 长连续序列 (Longest Continuous Increasing Subsequence)
8  * 问题描述:
9  * 给定一个未排序的整数数组, 找到最长连续序列的长度。
10 * <p>
11 * 要求算法的时间复杂度为  $O(n)$ 。
12 * <p>
13 * 示例 1:
14 * <p>
15 * 输入: [100, 4, 200, 1, 3, 2]
16 * 输出: 2
17 * 解释: 最长连续序列是 [100, 200], [1,2]。它的最大长度为 2。
18 * 示例 2:
19 * <p>
20 * 输入: [0, 3, 7, 2, 5, 8, 4, 6, 0, 1]
21 * 解释: 最长连续序列有 [0,3,7,8] [0,2,5,8]。它的最大长度为 4。
22 * 输出: 4
23 *
24 * @author xueancao.cxa on 2024/04/25
25 */
26 public class LongestIncreasingSubsequence {
27
28     public static void main(String[] args) {
29         int[] nums = {0, 3, 7, 2, 5, 8, 4, 6, 0, 1};
30         System.out.println("最长连续序列长度: " + lengthOfLIS(nums));
31     }
32
33     /**
34      * 最长连续子序列长度
35      * @param nums
36      * @return
37      */
38     public static int lengthOfLIS(int[] nums) {
39         if (nums == null || nums.length == 0) {
40             return 0;
41         }
42         int n = nums.length;
43         int[] dp = new int[n];
44         Arrays.fill(dp, 1);
45         for (int i = 1; i < n; i++) {
```

```
46         for (int j = 0; j < i; j++) {
47             if (nums[i] > nums[j]) {
48                 dp[i] = Math.max(dp[i], dp[j] + 1);
49             }
50         }
51     }
52     return Arrays.stream(dp).max().getAsInt();
53 }
54
55
56
57 }
```

最短路径

```
1 package com.alibaba.alsc.lemon.model.algo.dp;
2
3 import com.alibaba.fastjson.JSON;
4 import com.google.common.collect.Lists;
5
6 import java.util.ArrayList;
7 import java.util.LinkedList;
8 import java.util.List;
9 import java.util.Queue;
10 import java.util.stream.Collectors;
11
12 public class ShortestPathFinder {
13
14     /**
15      * ↖左上到右下最短路径
16      * @param args
17      */
18     public static void main(String[] args) {
19         int[][] inputData = {
20             {1, 3, 4},
21             {3, 4, 2},
22             {4, 2, 1}
23         };
24         int dis = minPathSumAndPath(inputData);
25         System.out.println("最短路径值: " + dis);
26     }
27
28     /**
29      * 计算最小路径和并返回路径
30      *
31      * @param grid 输入的网格
32      * @return 包含最小路径和和路径索引的数组
33      */
34     public static int minPathSumAndPath(int[][] grid) {
35         int m = grid.length;
36         int n = grid[0].length;
37
38         int[][] dp = new int[m][n];
39         dp[0][0] = grid[0][0];
40         // -1无, 0左, 1上, 2左上均可
41         int[][] pre = new int[m][n];
42         pre[0][0] = -1;
43
44         // 首行
45         for (int j = 1; j < n; j++) {
```

```

46         dp[0][j] = dp[0][j - 1] + grid[0][j];
47         pre[0][j] = -1;
48     }
49     // 首列
50     for (int i = 1; i < m; i++) {
51         dp[i][0] = dp[i - 1][0] + grid[i][0];
52         pre[i][0] = -1;
53     }
54
55     for (int i = 1; i < m; i++) {
56         for (int j = 1; j < n; j++) {
57             // 上, 左一致
58             if (dp[i - 1][j] == dp[i][j - 1]) {
59                 dp[i][j] = dp[i - 1][j] + grid[i][j];
60                 pre[i][j] = 2;
61             } else if (dp[i - 1][j] < dp[i][j - 1]) {
62                 // 上单元格
63                 dp[i][j] = dp[i - 1][j] + grid[i][j];
64                 pre[i][j] = 1;
65             } else if (dp[i - 1][j] > dp[i][j - 1]) {
66                 // 左单元格
67                 dp[i][j] = dp[i][j - 1] + grid[i][j];
68                 pre[i][j] = 0;
69             }
70         }
71     }
72     // 回溯, 找路径
73     List<List<int[]>> allPossiblePaths = findAllPossiblePaths(pre);
74     System.out.println("共有路径: " + allPossiblePaths.size());
75     allPossiblePaths.forEach(path -> {
76         System.out.println(JSON.toJSONString(path));
77     });
78     // 转换路径为整数数组
79     return dp[m - 1][n - 1];
80 }
81
82 public static List<List<int[]>> findAllPossiblePaths(int[][] pre) {
83     int m = pre.length;
84     int n = pre[0].length;
85     Queue<List<Integer>> queue = new LinkedList<>();
86
87     // 初始化末尾节点路径
88     List<Integer> endPath = new ArrayList<>();
89     endPath.add(m - 1);
90     endPath.add(n - 1);
91     queue.offer(endPath);
92
93     List<List<Integer>> allPaths = new ArrayList<>();

```

```

94
95     while (!queue.isEmpty()) {
96         List<Integer> currentPath = queue.poll();
97         int row = currentPath.get(currentPath.size() - 2);
98         int col = currentPath.get(currentPath.size() - 1);
99
100         // 如果到达左上角，添加路径到结果列表
101         if (row == 0 && col == 0) {
102             allPaths.add(new ArrayList<>(currentPath));
103             continue;
104         }
105         // 根据pre数组的值，尝试所有可能的路径
106         if (pre[row][col] == -1) {
107             if (col > 0) {
108                 addNextStep(queue, currentPath, row, col - 1);
109             }
110             if (row > 0) {
111                 addNextStep(queue, currentPath, row - 1, col);
112             }
113         } else if (pre[row][col] == 0) {
114             addNextStep(queue, currentPath, row, col - 1);
115         } else if (pre[row][col] == 1) {
116             addNextStep(queue, currentPath, row - 1, col);
117         } else if (pre[row][col] == 2) {
118             addNextStep(queue, currentPath, row, col - 1);
119             addNextStep(queue, currentPath, row - 1, col);
120         }
121     }
122     return allPaths.stream().map(path -> {
123         List<int[]> nodes = new ArrayList<>();
124         for (int i = 0; i < path.size(); ) {
125             int[] node = {path.get(i), path.get(i + 1)};
126             i += 2;
127             nodes.add(node);
128         }
129         return Lists.reverse(nodes);
130     }).collect(Collectors.toList());
131 }
132
133     private static void addNextStep(Queue<List<Integer>> queue, List<Integer> currentPath, int newRow, int newCol) {
134         List<Integer> newPath = new ArrayList<>(currentPath);
135         newPath.add(newRow);
136         newPath.add(newCol);
137         queue.offer(newPath);
138     }
139 }

```


背包问题

```

1 package com.alibaba.alisc.lemon.model.algo.dp;
2
3 /**
4  * 背包问题(物品可重复, 不限量)
5  * 给定两个整数数组 weights 和 values, 其中 weights[i] 是第 i 个物品的重量, value
6  *  s[i] 是第 i 个物品的价值。
7  * 另有一个最大容量 W 的背包。求解如何选择物品, 使得装入背包的物品总重量不超过 W 的情况
8  * 下, 价值最大化。
9  *
10  * @author xueancao.cxa on 2024/04/29
11  */
12 public class KnapsackFind {
13
14     public static void main(String[] args) {
15         // 重量
16         int[] weights = {1, 3, 4};
17         // 价值
18         int[] values = {1, 4, 5};
19         // 背包最大容量
20         int capacity = 5;
21         // 结果
22         int res = knapsackFind(weights, values, capacity);
23         System.out.println(res);
24     }
25
26     /**
27      * dp[i][w]:
28      * 0 1 2 3 4 5 <-背包容量w
29      * 0 = [0, 0, 0, 0, 0, 0]
30      * 1 = [0, 1, 2, 3, 4, 5]
31      * 2 = [0, 1, 2, 4, 5, 6]
32      * 3 = [0, 1, 2, 4, 5, 6]
33      *
34      * @param weights
35      * @param values
36      * @param capacity
37      * @return
38      */
39     private static int knapsackFind(int[] weights, int[] values, int capacity) {
40         // dp[i][w]表示 第i个物品放入容量为w的背包的最大价值
41         int[][] dp = new int[weights.length + 1][capacity + 1];
42         // 从第二个行, 第二列开始, 0行表示不放物品, 0列表示空间为0, 价值总和都是0
43         // i遍历物品

```

```

43     for (int i = 1; i <= weights.length; i++) {
44         // w遍历背包容量
45         for (int w = 1; w <= capacity; w++) {
46             // 第i个物品可以放下
47             if (weights[i - 1] <= w) {
48                 // 可以放n个i
49                 int n = w / weights[i - 1];
50                 // n个i的价值
51                 int x = n * values[i - 1];
52                 // 还有剩余空间
53                 if (w % weights[i - 1] > 0) {
54                     // +剩余空间可容纳的最大价值
55                     x += dp[i - 1][w % weights[i - 1]];
56                 }
57                 dp[i][w] = Math.max(dp[i - 1][w], x);
58             } else {
59                 dp[i][w] = dp[i - 1][w];
60             }
61         }
62     }
63     return dp[weights.length][capacity];
64 }
65 }

```

计算最小路径和并返回路径

```
1  import com.alibaba.fastjson.JSON;
2  import com.google.common.collect.Lists;
3
4  import java.util.ArrayList;
5  import java.util.LinkedList;
6  import java.util.List;
7  import java.util.Queue;
8  import java.util.stream.Collectors;
9
10 public class ShortestPathFinder {
11     /**
12      * 计算最小路径和并返回路径
13      *
14      * @param grid 输入的网格
15      * @return 包含最小路径和和路径索引的数组
16      */
17     public static int minPathSumAndPath(int[][] grid) {
18         int m = grid.length;
19         int n = grid[0].length;
20
21         int[][] dp = new int[m][n];
22         dp[0][0] = grid[0][0];
23         // -1无, 0左, 1上, 2左上均可
24         int[][] pre = new int[m][n];
25         pre[0][0] = -1;
26
27         // 首行
28         for (int j = 1; j < n; j++) {
29             dp[0][j] = dp[0][j - 1] + grid[0][j];
30             pre[0][j] = -1;
31         }
32         // 首列
33         for (int i = 1; i < m; i++) {
34             dp[i][0] = dp[i - 1][0] + grid[i][0];
35             pre[i][0] = -1;
36         }
37
38         for (int i = 1; i < m; i++) {
39             for (int j = 1; j < n; j++) {
40                 // 上, 左一致
41                 if (dp[i - 1][j] == dp[i][j - 1]) {
42                     dp[i][j] = dp[i - 1][j] + grid[i][j];
43                     pre[i][j] = 2;
44                 } else if (dp[i - 1][j] < dp[i][j - 1]) {
45                     // 上单元格
```

```

46         dp[i][j] = dp[i - 1][j] + grid[i][j];
47         pre[i][j] = 1;
48     } else if (dp[i - 1][j] > dp[i][j - 1]) {
49         // 左单元格
50         dp[i][j] = dp[i][j - 1] + grid[i][j];
51         pre[i][j] = 0;
52     }
53 }
54 }
55 List<List<int[]>> allPossiblePaths = findAllPossiblePaths(pre);
56 System.out.println("共有路径: " + allPossiblePaths.size());
57 allPossiblePaths.forEach(path -> {
58     System.out.println(JSON.toJSONString(path));
59 });
60 // 转换路径为整数数组
61 return dp[m - 1][n - 1];
62 }
63
64 public static List<List<int[]>> findAllPossiblePaths(int[][] pre) {
65     int m = pre.length;
66     int n = pre[0].length;
67     Queue<List<Integer>> queue = new LinkedList<>();
68
69     // 初始化末尾节点路径
70     List<Integer> endPath = new ArrayList<>();
71     endPath.add(m-1);
72     endPath.add(n-1);
73     queue.offer(endPath);
74
75     List<List<Integer>> allPaths = new ArrayList<>();
76
77     while (!queue.isEmpty()) {
78         List<Integer> currentPath = queue.poll();
79         int row = currentPath.get(currentPath.size() - 2);
80         int col = currentPath.get(currentPath.size() - 1);
81
82         // 如果到达右下角，添加路径到结果列表
83         if (row == 0 && col == 0) {
84             allPaths.add(new ArrayList<>(currentPath));
85             continue;
86         }
87         // 根据pre数组的值，尝试所有可能的路径
88         if (pre[row][col] == -1) {
89             if (col > 0) {
90                 addNextStep(queue, currentPath, row, col - 1);
91             }
92             if (row > 0) {
93                 addNextStep(queue, currentPath, row - 1, col);

```

```

94         }
95     } else if (pre[row][col] == 0) {
96         addNextStep(queue, currentPath, row, col - 1);
97     } else if (pre[row][col] == 1) {
98         addNextStep(queue, currentPath, row - 1, col);
99     } else if (pre[row][col] == 2) {
100         addNextStep(queue, currentPath, row, col - 1);
101         addNextStep(queue, currentPath, row - 1, col);
102     }
103 }
104 return allPaths.stream().map(path -> {
105     List<int[]> nodes = new ArrayList<>();
106     for (int i = 0; i < path.size(); ) {
107         int[] node = {path.get(i), path.get(i + 1)};
108         i += 2;
109         nodes.add(node);
110     }
111     return Lists.reverse(nodes);
112 }).collect(Collectors.toList());
113 }
114
115 private static void addNextStep(Queue<List<Integer>> queue, List<Integer> currentPath, int newRow, int newCol) {
116     List<Integer> newPath = new ArrayList<>(currentPath);
117     newPath.add(newRow);
118     newPath.add(newCol);
119     queue.offer(newPath);
120 }
121
122 public static void main(String[] args) {
123     int[][] inputData = {
124         {1, 3, 4},
125         {3, 4, 2},
126         {4, 2, 1}
127     };
128     int dis = minPathSumAndPath(inputData);
129     System.out.println("最短路径值: " + dis);
130 }
131 }

```


接雨水问题

标签：双指针

题目描述

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1:



▼ Plain Text

1

输入: height = [0,1,0,2,1,0,1,3,2,1,2,1]

2

输出: 6

3

解释: 上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

示例 2:

▼ Plain Text

1

输入: height = [4,2,0,3,2,5]

2

输出: 9

提示:

- $n == \text{height.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{height}[i] \leq 10^5$

解答

使用了双指针 **left** 和 **right** 分别从数组的两端向中间移动。

在移动过程中，用 **leftMax** 和 **rightMax** 分别记录左边和右边柱子的最大高度。如果当前柱子的高度小于最大高度，说明可以接到雨水，将雨水量累加到 **water** 中。

这个解法的时间复杂度为 $O(n)$ ，空间复杂度为 $O(1)$ 。

```
1 class Solution {
2     public int trap(int[] height) {
3         if (height == null || height.length == 0) {
4             return 0;
5         }
6
7         int left = 0, right = height.length - 1;
8         int leftMax = 0, rightMax = 0;
9         int water = 0;
10
11        while (left < right) {
12            if (height[left] < height[right]) {
13                if (height[left] >= leftMax) {
14                    leftMax = height[left];
15                } else {
16                    water += leftMax - height[left];
17                }
18                left++;
19            } else {
20                if (height[right] >= rightMax) {
21                    rightMax = height[right];
22                } else {
23                    water += rightMax - height[right];
24                }
25                right--;
26            }
27        }
28
29        return water;
30    }
31 }
```

最长连续子序列长度

Java

```
1 public class LongestIncreasingSubsequence {
2
3     /**
4      * 最长连续子序列长度
5      * @param nums
6      * @return
7      */
8     public static int lengthOfLIS(int[] nums) {
9         if (nums == null || nums.length == 0) {
10             return 0;
11         }
12         int n = nums.length;
13         int[] dp = new int[n];
14         Arrays.fill(dp, 1);
15         for (int i = 1; i < n; i++) {
16             for (int j = 0; j < i; j++) {
17                 if (nums[i] > nums[j]) {
18                     dp[i] = Math.max(dp[i], dp[j] + 1);
19                 }
20             }
21         }
22         return Arrays.stream(dp).max().getAsInt();
23     }
24
25     public static void main(String[] args) {
26         int[] nums = {0, 3, 7, 2, 5, 8, 4, 6, 0, 1};
27         System.out.println("最长连续序列长度: " + lengthOfLIS(nums));
28     }
29
30 }
```

53. 最大子序和

最大子序和

题目描述

给定一个整数数组 `nums`，找到一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

示例

▼

Plain Text

1

输入：[-2,1,-3,4,-1,2,1,-5,4],

2

输出：6

3

解释：连续子数组 [4,-1,2,1] 的和为 6。

进阶

如果你已经实现一个时间复杂度为 $O(n)$ 的解决方案，尝试使用更高级的数据结构来实现 $O(1)$ 空间复杂度的解决方案。

解法

```
1 public class Solution {  
2     public int maxSubArray(int[] nums) {  
3         // 初始化最大和为负无穷大，因为数组中的数可能都是负数  
4         int maxSum = Integer.MIN_VALUE;  
5         // 初始化当前和为0  
6         int currentSum = 0;  
7  
8         // 遍历数组  
9         for (int num : nums) {  
10            // 将当前元素加到当前和上  
11            currentSum = currentSum + num;  
12            // 如果当前和大于最大和，则更新最大和  
13            maxSum = Math.max(maxSum, currentSum);  
14            // 如果当前和变为负数，则重置为0，因为当前子数组的和不可能是最大和  
15            currentSum = Math.max(currentSum, 0);  
16        }  
17  
18        // 返回最大和  
19        return maxSum;  
20    }  
21 }
```

维护一个当前子数组的和，如果当前子数组的和变为负数，则意味着它不会对后续子数组和产生贡献，因此可以重置为0。同时，不断更新最大子数组和，直到遍历完整个数组。时间复杂度是 $O(n)$ ，因为每个元素只被遍历一次。

最长回文子串

标签：动态规划

给你一个字符串 s ，找到 s 中最长的回文子串

如果字符串的反序与原始字符串相同，则该字符串称为回文字符串。

示例 1:

	Plain Text
1	输入: $s = \text{"babad"}$
2	输出: "bab"
3	解释: "aba" 同样是符合题意的答案。

示例 2:

	Plain Text
1	输入: $s = \text{"cbbd"}$
2	输出: "bb"

提示:

- $1 \leq s.length \leq 1000$
- s 仅由数字和英文字母组成

解答

首先初始化一个二维布尔数组 dp 来记录子串是否是回文串。然后，从字符串的末尾向前逐个字符遍历，对比标 j 从 i 开始到字符串末尾，判断从 i 到 j 的子串是否是回文串

在遍历过程中，判断条件是如果当前字符相等，并且子串长度小于等于2 ($j - i \leq 2$) 或者子串的内部子串是回文串 ($dp[i + 1][j - 1]$ 是 $true$)，那么当前子串也是回文串，将 $dp[i][j]$ 设置为

true。

同时，记录最长回文子串的起始位置和结束位置。当找到更长的回文子串时，更新 **start** 和 **end**。

最后返回起始位置到结束位置的子串，即最长回文子串。

这种动态规划的思路正确地利用了子问题的信息来构建更长的回文串，时间复杂度为 $O(n^2)$ 。

```
1 class Solution {
2     public String longestPalindrome(String s) {
3         if (s == null || s.length() < 1) {
4             return "";
5         }
6
7         int n = s.length();
8         boolean[][] dp = new boolean[n][n];
9         int start = 0;
10        int end = 0;
11
12        // 逆向遍历字符串，移动标i
13        for (int i = n - 1; i >= 0; i--) {
14            // 对比标j
15            for (int j = i; j < n; j++) {
16                if (s.charAt(i) == s.charAt(j) && (j - i <= 2 || dp[i + 1]
17                    [j - 1])) {
18                    dp[i][j] = true;
19
20                    if (j - i > end - start) {
21                        start = i;
22                        end = j;
23                    }
24                }
25            }
26
27            return s.substring(start, end + 1);
28        }
29    }
```

2Sum、3Sum、4Sum问题

三数之和（3Sum）和四数之和（4Sum）是在数组中查找特定和的问题，通常采用不同的方法来解决。

两数之和（2Sum）问题：

问题描述： 给定一个包含n个整数的数组 `nums`，判断 `nums` 中是否存在2个元素a, b，使得 `a + b = target`。找到所有满足条件且不重复的两元组。

解决方案： 使用hash表解决

```
1  import java.util.HashMap;
2  import java.util.Map;
3
4  public class Solution {
5      public int[] twoSum(int[] nums, int target) {
6          Map<Integer, Integer> map = new HashMap<>();
7
8          for (int i = 0; i < nums.length; i++) {
9              int complement = target - nums[i];
10             if (map.containsKey(complement)) {
11                 return new int[]{map.get(complement), i};
12             }
13             map.put(nums[i], i);
14         }
15
16         return new int[0];
17     }
18 }
19
```

三数之和（3Sum）问题：

问题描述： 给定一个包含n个整数的数组 `nums` ，判断 `nums` 中是否存在三个元素a, b, c, 使得 `a + b + c = target` 。找到所有满足条件且不重复的三元组。

解决方案：

1. 首先对数组排序。
2. 遍历数组，对每个元素 `nums[i]` ，使用双指针在其后部分查找两个数的和等于 `-nums[i]` ，即从左右两端向中间靠拢的方式。
3. 使用双指针可以在O(n)的时间内找到所有满足条件的三元组，同时避免了重复的组合。

以下是Java示例代码：

```
1  import java.util.*;
2
3  public class Solution {
4      public List<List<Integer>> threeSum(int[] nums, int target) {
5          List<List<Integer>> result = new ArrayList<>();
6          Arrays.sort(nums);
7
8          for (int i = 0; i < nums.length - 2; i++) {
9              if (i > 0 && nums[i] == nums[i - 1]) {
10                 continue; // 避免重复元素
11             }
12             int left = i + 1;
13             int right = nums.length - 1;
14             while (left < right) {
15                 int sum = nums[i] + nums[left] + nums[right];
16                 if (sum == target) {
17                     result.add(Arrays.asList(nums[i], nums[left], nums[right]));
18                     while (left < right && nums[left] == nums[left + 1]) {
19                         left++;
20                     }
21                     while (left < right && nums[right] == nums[right - 1]) {
22                         right--;
23                     }
24                     left++;
25                     right--;
26                 } else if (sum < target) {
27                     left++;
28                 } else {
29                     right--;
30                 }
31             }
32         }
33
34         return result;
35     }
36 }
```

四数之和 (4Sum) 问题:

问题描述： 给定一个包含n个整数的数组 `nums` 和一个目标值 `target`，找出 `nums` 中所有可以使四个整数相加等于 `target` 的不重复组合。

解决方案：

1. 首先对数组排序。
2. 使用四重循环固定前两个元素（`nums[i]` 和 `nums[j]`），然后使用双指针在剩余部分中查找两个数的和等于 `target - nums[i] - nums[j]`，即从左右两端向中间靠拢的方式。
3. 使用双指针可以在 $O(n^2)$ 的时间内找到所有满足条件的四元组，同时避免了重复的组合。

以下是Java示例代码：

```
1  import java.util.*;
2
3  public class Solution {
4      public List<List<Integer>> fourSum(int[] nums, int target) {
5          List<List<Integer>> result = new ArrayList<>();
6          Arrays.sort(nums);
7
8          for (int i = 0; i < nums.length - 3; i++) {
9              if (i > 0 && nums[i] == nums[i - 1]) {
10                 continue; // 避免重复元素
11             }
12             for (int j = i + 1; j < nums.length - 2; j++) {
13                 if (j > i + 1 && nums[j] == nums[j - 1]) {
14                     continue; // 避免重复元素
15                 }
16                 int left = j + 1;
17                 int right = nums.length - 1;
18                 while (left < right) {
19                     int sum = nums[i] + nums[j] + nums[left] + nums[right]
;
20                     if (sum == target) {
21                         result.add(Arrays.asList(nums[i], nums[j], nums[le
ft], nums[right]));
22                     while (left < right && nums[left] == nums[left + 1
]) {
23                         left++;
24                     }
25                     while (left < right && nums[right] == nums[right -
1]) {
26                         right--;
27                     }
28                     left++;
29                     right--;
30                 } else if (sum < target) {
31                     left++;
32                 } else {
33                     right--;
34                 }
35             }
36         }
37     }
38
39     return result;
40 }
41 }
```

这两个问题的解决方案使用了类似的思路，即对数组排序并使用双指针技巧来查找满足条件的组合，因此时间复杂度都在 $O(n^2)$ 级别。